# Python Programming: From Basics to Advanced

## Introduction to Python

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and released in 1991. Its design philosophy emphasises code readability and simplicity, making it one of the most popular languages worldwide.

Python supports multiple paradigms: procedural, object-oriented, and functional programming. It runs on Windows, macOS, Linux, and can even run in the browser via WebAssembly.

## Core Data Types

Python's built-in types include int, float, complex, bool, str, bytes, list, tuple, set, frozenset, and dict. Dynamic typing means variables do not need explicit type declarations, though type hints (PEP 484) are encouraged for large codebases.

Lists are mutable ordered sequences; tuples are immutable. Dictionaries map unique keys to values and maintain insertion order since Python 3.7.

## Functions and Modules

Functions are defined with the 'def' keyword. Python supports default arguments, keyword arguments, *args, **kwargs, and lambda expressions for anonymous functions.

Modules group related code into files; packages group modules into directories. The Python Package Index (PyPI) hosts over 500,000 packages installable via pip.

## Object-Oriented Programming

Classes are defined with the 'class' keyword. Python supports single and multiple inheritance, mixins, abstract base classes (abc module), dataclasses, and metaclasses.

Dunder (double-underscore) methods like __init__, __str__, __repr__, __len__, and __iter__ let you integrate custom objects with Python's built-in operations.

## Popular Libraries

Data Science: NumPy (arrays), Pandas (dataframes), Matplotlib & Seaborn (visualisation).

Machine Learning: scikit-learn, TensorFlow, PyTorch, Keras, XGBoost.

Web Development: Django (batteries-included), Flask (micro-framework), FastAPI (async).

Automation: Selenium, Playwright, Beautiful Soup, Scrapy.

DevOps: Ansible, Fabric, Paramiko.

## Best Practices

Follow PEP 8 style guidelines. Use virtual environments (venv or conda). Write unit tests with pytest or unittest. Document code with docstrings. Use type hints for clarity.

Favour composition over inheritance, keep functions small and focused, and rely on Python's built-in data structures before reaching for external libraries.